

CHAPTER I

INTRODUCTION

1.1 Introduction

One of the key areas where Internet of Things (IoT) is currently seeing increased adoption is large-scale, remote monitoring applications. The ability to connect a large number of devices spread out over vast areas can provide significant benefits. In these remote applications, IoT technologies can be used in a wireless sensor network (WSN) to register environmental phenomena: from soil and snow monitoring across large nature reserves , to plant stress monitoring across large agricultural fields . These use cases typically require only an extremely small throughput rate and are not latency critical. To maximize the lifespan of the IoT nodes, which are expected to operate on a limited energy supply for several years, it is crucial to optimize both their hardware and software for minimal power consumption . Recently, a few wireless Internet of Things (IoT) technologies have proven their worth in actual deployments. The Long-range wide-area network (LoRa WAN) implementation in particular has become widespread. LoRa WAN is designed specifically for low-throughput communication and offers long-range, low-power, and unlicensed spectrum operation. In these networks, a gateway is connected to the cloud using a single-hop star network topology, as is typical for most IoT networks. Despite the 2–5 km range of coverage that Lora WAN can accomplish [4], it might not be sufficient to cover the hundreds of square kilometers required in the aforementioned use scenarios, or when propagation is severely attenuated, e.g., because of obstructed paths. Coverage could be provided by distributing a large amount of long-range wide-area network (LoRa WAN) gateways in the area; however, this proves to be unfeasible due to lacking (cellular) network coverage, energy restrictions, or financial cost. Therefore, we propose a long range (LoRa)-based multi-hop network to extend the coverage, specifically optimized for energy efficiency.

A multi-hop protocol allows messages to be transmitted from one node to another through a series of intermediate nodes, effectively allowing packets to *hop* from one node to the next until they reach their destination. This enables the network to cover a much larger area than would be possible with a single hop. In the implementation presented in this work, we assume

only one possible destination: the gateway or the sink, which can store the gathered data or relay them to another cloud-connected network.

1.2 Problem Statement

Efficient and reliable energy demand forecasting is critical for the optimal operation and sustainability of modern smart grids. Traditional energy consumption prediction systems often face challenges such as limited data coverage, unreliable communication in remote or large-scale sensor deployments, and insufficient integration of external environmental factors that significantly influence energy demand patterns. Single-hop wireless communication technologies like conventional LoRa have inherent range and reliability limitations, which result in incomplete or delayed data acquisition from distributed IoT sensors. Furthermore, existing machine learning approaches for demand classification often struggle with capturing complex, nonlinear relationships in heterogeneous datasets, leading to suboptimal prediction accuracy. Addressing these challenges requires a communication network design capable of extending coverage while maintaining data integrity, alongside advanced predictive models that effectively leverage both sensor data and external contextual information.

1.3 Motivation

The growing adoption of IoT sensors in energy management systems offers unprecedented opportunities to collect granular real-time data, yet the potential remains underutilized due to communication constraints and modeling limitations. The motivation behind this work stems from the need to bridge the gap between raw sensor data acquisition and actionable predictive insights to support smart grid operations. Employing a multi-hop LoRa network architecture allows for overcoming communication barriers posed by wide geographical sensor deployments, enabling continuous and reliable data flow. At the same time, integrating advanced machine learning models like Random Forest Classifier enables capturing complex patterns in energy consumption influenced by multifaceted environmental factors. By combining robust network design with sophisticated analytics, the objective is to enhance the accuracy and scalability of energy demand prediction, contributing towards smarter, more sustainable energy utilization and management.

1.4 Objective

The primary objective is to develop a scalable and reliable IoT-based framework for predictive energy demand modeling that integrates multi-hop LoRa wireless communication and advanced classification algorithms. Specific objectives include: designing and implementing a multi-hop LoRa communication network that extends coverage and improves data transmission

reliability; conducting exploratory data analysis to identify critical features and relationships between IoT sensor data and external environmental variables; evaluating and comparing the performance of Support Vector Classifier and Random Forest Classifier in classifying energy demand levels; and ultimately deploying an optimized predictive model that delivers improved accuracy, robustness, and generalization for real-time energy demand forecasting in smart grid environments.

1.5 Significance

The significance of the work lies in addressing two key challenges in IoT-enabled energy demand forecasting: communication reliability and predictive accuracy. By adopting a multi-hop LoRa architecture, the framework ensures enhanced coverage and reduced data loss in large-scale sensor networks, which is essential for continuous monitoring and timely decision-making. The application of Random Forest Classifier, with its ability to handle complex nonlinearities and reduce overfitting, improves the predictive power over traditional classifiers like SVC, leading to more dependable energy demand forecasts. This integration paves the way for smarter energy resource management, enabling utilities and grid operators to optimize energy distribution, reduce wastage, and better respond to dynamic consumption patterns. Ultimately, the research supports the broader goals of energy sustainability, grid stability, and cost-effective infrastructure management within the evolving landscape of smart cities and IoT ecosystems.

1.6 Applications

Real-time energy demand forecasting for smart grid operators to optimize load balancing and reduce outages.

Enhanced monitoring of distributed energy resources in remote or wide-area network through multi-hop LoRa communication.

Integration into smart home and building automation systems for predictive energy management and cost savings.

Support for renewable energy integration by predicting demand fluctuations influenced by weather and environmental conditions.

CHAPTER II

LITERATURE SURVEY

Dafflon et al. developed a distributed temperature profiling system for acquiring vertically and laterally dense temperature data in soil and snow. Their system enables high-resolution thermal monitoring, crucial for understanding subsurface dynamics, which supports accurate environmental modeling. Such temperature sensing mechanisms can be leveraged in smart energy systems for environmental context-aware predictions. The framework provides insights into sensor deployment strategies in LoRa-enabled monitoring systems. Geldhof et al. proposed a digital leaf movement sensor to monitor plant stress responses in real time. Their work bridges plant physiology and electronics, showcasing the value of smart sensing in detecting environmental stressors. The sensor's data acquisition principles highlight how real-time monitoring can inform predictive algorithms, applicable to LoRa-based agricultural energy forecasting systems. This also underscores the integration of biological signals into IoT frameworks. Callebaut et al. discussed power optimization strategies and technologies crucial for designing long-battery-life IoT devices. Their work provides a detailed exploration of energy-saving techniques, from hardware-level innovations to software strategies. These concepts are foundational for sustaining LoRa-based networks in remote deployments. Efficient energy usage directly enhances system longevity and reliability in energy monitoring applications. Petajajarvi et al. performed a range evaluation of LPWANs, especially LoRa, focusing on coverage and signal attenuation models. Their findings help inform optimal node placement and transmission range estimations in outdoor environments. Such knowledge supports the scalability and deployment planning of predictive energy systems based on LoRa. Accurate modeling of range is critical for data reliability in sparse regions. Rochester et al. implemented lightweight carrier sensing mechanisms in LoRa networks to assess performance impacts. Their results indicate improvements in channel access efficiency without sacrificing energy savings. The protocol enhancements support lower latency, which benefits time-sensitive energy demand prediction. Their work highlights the importance of MAC-level innovations for practical LoRa applications.

Aslam et al. explored multi-hop LoRa communications for green smart cities, emphasizing network sustainability. Their architecture balances energy efficiency and coverage, essential for large-scale IoT systems. The study underlines the potential of LoRa in urban energy infrastructures, enabling efficient demand data transmission. The multi-hop concept ensures consistent network coverage for real-time predictions. Sciullo et al. designed LOCATE, a

mobile emergency management system using LoRa. Their solution facilitates rapid data dissemination during emergencies, proving LoRa's robustness in dynamic environments. The adaptive design strategy informs the architecture of resilient LoRa networks for continuous energy data collection. Ensuring connectivity under pressure scenarios is key for reliable energy prediction. Sartori et al. enabled RPL-based multi-hop communications over LoRa networks, enhancing routing efficiency. Their integration of standard IoT protocols with LoRa broadens compatibility and deployment options. Efficient routing minimizes energy waste in sensor nodes, contributing to prolonged system operation in remote energy monitoring. Their work supports IoT scalability in energy infrastructure. Zhou et al. presented LoRa-Hybrid, a multi-hop LoRa WAN-based solution tailored for regional microgrids. Their system improves connectivity and data collection in distributed energy systems. LoRa-Hybrid's architecture aligns well with predictive energy demand models needing consistent updates from edge nodes. The hybrid approach ensures flexibility and redundancy in communication. Prade et al. developed a multi-radio, multi-hop LoRa communication model suited for large-scale IoT systems. Their architecture enhances robustness and data throughput across expansive deployment areas. Such scalable communication designs are ideal for national-level energy demand monitoring using LoRa. The model encourages fault-tolerant transmission in predictive systems. Leonardi et al. introduced MRT-LoRa, a real-time communication protocol tailored for industrial IoT using LoRa. It guarantees low-latency data flow with time constraints, crucial for industrial energy management. Their protocol aligns well with smart grids requiring timely data for accurate energy demand forecasting. The emphasis on timing synchronization supports real-time analytics.

Zhu et al. improved mesh LoRa capacity by applying spreading-factor-based network clustering. Their method enhances throughput and reduces interference, essential for high-density sensor networks. This directly supports predictive energy systems operating in urban environments. The clustering strategy enhances network stability and prediction accuracy. Piyare et al. introduced On-demand LoRa, an asynchronous TDMA-based protocol designed for energy efficiency and low latency. The solution significantly reduces idle listening and improves node battery life. Such MAC-level improvements are crucial for deploying long-term energy monitoring systems. The protocol's asynchronous nature is ideal for event-driven prediction tasks. Bezunartea et al. investigated methods for enhancing energy efficiency in LoRa multi-hop networks. They evaluated transmission policies and node behaviors that contribute to minimal power usage. Their findings are relevant for optimizing LoRa-based energy monitoring systems in remote or inaccessible areas. Their protocol design supports

predictive models with consistent, energy-efficient data input. Abrar do et al. implemented a linear multi-hop LoRa network for monitoring underground medieval aqueducts. Their project highlighted LoRa's effectiveness in subterranean and structurally challenging environments. Insights from this deployment can be translated to smart underground energy pipelines or tunnels. Reliable data acquisition in constrained environments supports safer energy infrastructure management. Abrar do et al. continued their work by applying LoRa for monitoring black powder flow in pipelines. They leveraged a multi-hop approach to maintain signal integrity across metal-dense environments. This application is particularly useful for industrial energy systems requiring predictive failure detection. The study proves LoRa's industrial-grade applicability. Escobar et al. proposed the JMAC protocol—a cross-layer approach optimized for multi-hop LoRa networks. It synchronizes MAC and network layers to achieve high efficiency in energy-constrained settings. This design philosophy is beneficial for LoRa-based energy forecasting systems requiring minimal communication overhead. The reduced complexity aids in scaling IoT deployments. Mai et al. introduced a latency-minimized multi-hop LoRa protocol tailored for real-time applications. Their system adapts route selection based on latency constraints, vital for predictive analytics. In energy demand prediction, delayed data can severely impact forecast accuracy. Their work supports time-sensitive deployments in smart energy systems.

Adame Vázquez et al. proposed HARE, an efficient multi-hop uplink communication framework for self-organizing LPWANs. Their system emphasizes autonomous routing and dynamic topology changes. This approach supports scalable energy prediction systems that adapt to infrastructure growth. The self-organizing feature reduces maintenance in energy-sensitive deployments. Tran et al. implemented a two-hop real-time LoRa protocol tailored for industrial control. Their protocol supports delay-bounded communication for automated processes. This aligns with the need for timely energy consumption data in predictive models. Their work enhances LoRa's reliability in real-world energy infrastructures. Tran et al. extended their protocol into a dynamic multi-hop real-time LoRa system. This version supports changing network topologies, enabling robust predictive systems in fluctuating environments. Their approach ensures continuous data acquisition critical for real-time energy analytics. The model is adaptive to IoT dynamics found in energy grids. Bachir et al. offered foundational insights into MAC layer essentials in wireless sensor networks. Their review establishes the principles for efficient communication applicable to LoRa-based IoT systems. Understanding MAC protocols is essential for optimizing data flow in energy monitoring networks. Their work supports designing low-latency and energy-aware systems. Iqbal et al. evaluated multi-

hop uplink strategies using LoRa in LPWAN environments. They provided a performance assessment based on varying hop counts and network conditions. Their work contributes to route planning and reliability improvements in energy forecasting networks. Multi-hop uplink ensures data continuity in wide-area monitoring. Cotrim et al. conducted a review and classification of LoRa WAN mesh networks for multi-hop communication. Their taxonomy outlines potential architectures suitable for energy systems. Understanding different mesh topologies aids in selecting the right framework for energy demand prediction. Their review consolidates best practices and performance considerations. Leenders et al. developed an energy-efficient multi-hop LoRa protocol utilizing preamble sampling. Their technique significantly reduces listening time and improves battery longevity. This innovation is beneficial for long-term energy demand monitoring where sensor replacement is impractical. The protocol ensures minimal energy consumption while maintaining data accuracy.

CHAPTER III

EXISTING SYSTEM

3.1 Traditional System

Step 1: Sensor Deployment and Data Collection: In the traditional setup, IoT sensors are strategically installed across the energy infrastructure to monitor parameters such as voltage, current, frequency, power consumption, and sometimes basic environmental factors like temperature and humidity. These sensors are responsible for collecting raw data at regular intervals. The sensors operate autonomously and are programmed to log real-time data reflecting the energy usage patterns of different points in the system. However, this stage is strictly limited to data acquisition and storage without any intelligent filtering or analysis at the edge.

Step 2: Single-Hop Wireless Communication: Once data is collected, it is transmitted from each sensor node to a central gateway using single-hop wireless protocols, commonly standard LoRa or similar low-power wide-area communication technologies. In this architecture, each sensor must directly communicate with the gateway without any intermediate relaying nodes. This setup works well when sensors are within the communication range, but the network becomes less effective in large-scale or geographically dispersed deployments. Signal attenuation, interference, and physical obstructions can significantly affect the reliability of this communication model.

Step 3: Centralized Data Aggregation and Storage: The gateway acts as the central hub, receiving data from all the connected sensor nodes. It aggregates the collected information into a centralized database, where data is stored for later retrieval and processing. This centralized model simplifies the architecture but introduces bottlenecks when handling high-volume or high-frequency data. Additionally, the system lacks any form of distributed intelligence, relying completely on the central gateway for processing and decision-making.

Step 4: Rule-Based or Statistical Analysis for Demand Estimation: Traditionally, energy demand forecasting in such systems is done using rule-based approaches or basic statistical methods. These techniques involve applying predefined rules, such as thresholds for identifying peak load periods or using historical averages to estimate future consumption. Although easy to implement, these methods are inherently static and fail to adapt to changing conditions or user behaviors. They do not learn from patterns or dynamically adjust to new data, which limits the system's forecasting capabilities.

Step 5: Limited Integration of External Environmental Factors: In most traditional systems, external contextual factors like weather conditions, seasonal changes, or temperature fluctuations are either entirely ignored or manually fed into the system in an offline manner. There is no real-time or automated integration of such external data with sensor readings. As a result, the forecasting process does not consider important correlations between external variables and energy consumption, reducing the accuracy and usefulness of the insights generated.

Step 6: Basic Reporting and Alert Mechanisms: After the rule-based processing is complete, the system generates periodic reports summarizing the energy consumption trends, potential threshold violations, or unusual usage patterns. Alerts are triggered only when predefined conditions are breached, such as a sensor reading exceeding a set value. These alerts are then forwarded to operators for manual inspection or intervention. However, these reports are reactive rather than proactive and often come with delays due to centralized processing and limited analytical depth.

3.2 Limitations

Limited Network Coverage: Single-hop communication restricts sensor nodes to be within direct range of the gateway, limiting deployment flexibility and coverage area.

Data Loss and Reliability Issues: In remote or obstructed environments, single-hop transmissions suffer from high packet loss, leading to incomplete or unreliable data collection.

Scalability Constraints: Adding more sensor nodes increases communication congestion and reduces performance due to limited direct connectivity to the gateway.

Static and Inaccurate Demand Forecasting: Rule-based or statistical forecasting methods fail to capture complex nonlinear relationships in energy consumption, resulting in suboptimal predictions.

Lack of Real-Time Adaptability: Traditional systems are slow to respond to sudden changes in consumption patterns or environmental conditions, impacting decision-making accuracy.

Minimal Integration of External Environmental Factors: Insufficient use of contextual external data limits the ability to understand and predict demand variations effectively.

Centralized Architecture Vulnerabilities: The reliance on a single gateway creates a single point of failure and potential bottleneck in data processing.

Limited Predictive Analytics: Absence of machine learning or advanced analytical techniques means forecasting is often reactive rather than proactive.

CHAPTER IV

PROPOSED SYSTEM

4.1 Overview

The proposed system integrates a multi-hop LoRa-based wireless communication framework with machine learning techniques to enable efficient and intelligent energy demand modeling using IoT sensor data. Unlike traditional single-hop communication, the multi-hop architecture enhances network coverage and reliability by allowing data to be transmitted through intermediate sensor nodes. The system collects real-time energy consumption data along with environmental factors such as temperature and humidity, which are preprocessed and enriched for predictive analysis. A Support Vector Classifier (SVC) is initially used as a baseline, while a Random Forest Classifier (RFC) is proposed to improve accuracy and robustness in forecasting energy demand patterns. The system performs continuous monitoring, intelligent prediction, and early detection of abnormal consumption trends, thereby supporting proactive energy management and optimized decision-making.

Step 1: Advanced IoT Sensor Deployment and Data Acquisition: The proposed system begins with a refined deployment of IoT sensor nodes capable of monitoring diverse energy-related parameters such as voltage, current, power consumption, as well as environmental factors like ambient temperature, humidity, and light intensity. These smart sensors are enhanced to handle multi-dimensional data, capturing both energy usage and contextual environmental variables that can influence demand. The sensors are distributed across multiple levels of the energy network—residential, commercial, or industrial zones—ensuring a more granular and wide-scale coverage of energy consumption behavior.

Step 2: Multi-Hop LoRa-Based Communication Architecture: Unlike the traditional single-hop model, the proposed system employs a multi-hop LoRa-based wireless communication protocol. This allows data packets from sensor nodes that are out of the direct communication range of the central gateway to be relayed through intermediate nodes. Each sensor acts as both a data source and a routing node, facilitating robust and extended-range communication. This mesh-like structure enhances the network's reliability, scalability, and fault tolerance. The multi-hop topology significantly reduces packet loss and ensures continuous transmission even in complex terrains or high-density deployments.

Step 3: Real-Time Data Aggregation and Preprocessing: As data from sensor nodes is transmitted through the multi-hop network to the central gateway, it is immediately aggregated and subjected to real-time preprocessing. This preprocessing includes null value checking,

handling missing data, encoding categorical variables using label encoding, and normalization of numerical values. Noise filtering techniques are applied to remove outliers and irrelevant entries. This step ensures the integrity and quality of the data before it is passed on to the machine learning pipeline for training or prediction.

Step 4: Integration of External Environmental and Contextual Factors: The system enhances its prediction capability by dynamically integrating external datasets such as weather conditions (temperature, wind speed, solar radiation), day-of-week, holidays, or seasonal information from APIs or local sources. These variables are aligned temporally with the IoT sensor data to create a comprehensive feature set. By incorporating these external factors, the model captures hidden correlations between environmental changes and energy demand behavior—something not possible in traditional approaches.

Step 5: Exploratory Data Analysis (EDA): Before applying machine learning, an extensive Exploratory Data Analysis (EDA) is performed to understand data distributions, correlations, and patterns. Visualization tools such as count plots, histograms, and heatmaps are used to explore class distributions, feature importance, and outlier behavior. EDA also reveals insights into time-based demand variations and uncovers hidden relationships between environmental conditions and power usage. This step informs the feature selection process and helps optimize model performance.

Step 6: Machine Learning Model Training and Evaluation (SVC and RFC): For the energy demand classification task, the system initially employs a Support Vector Classifier (SVC) as the baseline model. SVC is chosen for its effectiveness in handling high-dimensional feature spaces and class boundaries. To improve performance, a Random Forest Classifier (RFC) is proposed and implemented as the primary predictive model. RFC's ensemble learning mechanism provides better accuracy, robustness to overfitting, and superior handling of non-linear data. The models are trained using historical and real-time sensor data enriched with environmental factors. Performance metrics such as accuracy, precision, recall, and F1-score are used to evaluate and compare the classifiers.

Step 7: Predictive Energy Demand Forecasting and Intelligent Decision Support: Once trained, the machine learning model is deployed to perform real-time predictive analysis on incoming data streams. The system continuously forecasts short-term energy demand, classifies consumption patterns, and identifies abnormal or peak-demand conditions. These predictions support proactive energy distribution, load balancing, and dynamic grid adjustments. Decision-makers can use this intelligent feedback to optimize resource allocation and prevent overloading, leading to improved operational efficiency and cost-effectiveness.

Step 8: Visualization and Alerting System: The final stage includes the visualization of predicted energy demand and anomalies through dashboards. Operators can view real-time trends, prediction accuracy, and decision insights. Additionally, if the system detects potential overconsumption or irregular patterns, it automatically triggers intelligent alerts and recommendations for preventive action. This allows utility providers to maintain energy reliability, prevent outages, and support demand-side energy management strategies.

4.2 Data Preprocessing

Step 1: Data Collection and Integration: The data preprocessing phase begins with the collection of raw datasets from various IoT sensors deployed across the energy monitoring network. These sensors generate data on parameters like voltage, current, and power consumption, along with environmental factors such as temperature and humidity. Additionally, external contextual data such as weather reports and time-based indicators (e.g., time of day, day of the week) are gathered from trusted APIs or local systems. These diverse datasets are then integrated into a unified format, ensuring that every entry is properly time-synchronized and aligned with the correct sensor node ID. This step lays the foundation for accurate analysis by consolidating all relevant inputs into a single structured dataset.

Step 2: Handling Missing and Null Values: After integration, the dataset is examined for missing or null values, which are common due to sensor glitches, connectivity drops, or environmental interference. Instead of simply discarding rows with missing data, intelligent imputation techniques are employed. For numerical features, missing values are filled using statistical methods such as mean, median, or mode imputation based on the distribution. For categorical features (if any), the most frequent or contextually appropriate value is used. This step ensures that no useful information is lost while maintaining data consistency and integrity.

Step 3: Outlier Detection and Removal: To improve the quality of the dataset and prevent skewed model performance, outliers are detected and managed. Outliers are identified using techniques such as the interquartile range (IQR) method or Z-score analysis. These are data points that fall significantly outside the normal range and may be due to faulty sensor readings or extreme environmental interference. Depending on the context, outliers are either removed or capped (i.e., replaced with boundary values). This step is essential to reduce noise and enhance the model's ability to learn accurate patterns.

Step 4: Label Encoding of Categorical Variables: If the dataset includes any categorical features such as day names (Monday, Tuesday, etc.) or weather conditions (Sunny, Cloudy, etc.) they must be converted into numerical form for compatibility with machine learning

algorithms. Label encoding is used to assign a unique integer value to each category. This method is computationally efficient and maintains the order of labels if required. Encoding categorical features transforms them into usable inputs that the classifiers (SVC and RFC) can effectively interpret during training and inference.

Step 5: Feature Scaling and Normalization: Since the dataset includes features with different units and scales (e.g., voltage in volts, temperature in °C), normalization or standardization is performed to bring them to a uniform scale. Min-Max normalization is often used to rescale features to a 0–1 range, while standardization converts features to have a mean of 0 and a standard deviation of 1. This step is particularly important for models like SVC that are sensitive to the magnitude of feature values. Scaling ensures that no single feature disproportionately influences the learning process.

Step 6: Feature Engineering and Selection: In this step, new features are engineered by combining or transforming existing ones to enhance the predictive power of the model. For example, time-based features such as peak-hour flags, seasonal indicators, or rolling averages of consumption can be derived. Correlation analysis is also conducted to identify the most relevant features that contribute significantly to energy demand. Irrelevant or redundant features are dropped to reduce computational complexity and avoid overfitting. This curated feature set is then finalized for model training.

Step 7: Splitting the Dataset for Training and Testing: The final preprocessed dataset is split into training and testing sets, typically using an 80-20 or 70-30 ratio. This separation ensures that the model is trained on one portion of the data and evaluated on an unseen portion, providing an unbiased estimate of performance. Cross-validation techniques may also be applied to ensure that the model generalizes well across different data subsets. This step marks the end of preprocessing and prepares the data for the machine learning modeling phase.

4.3 EDA

Step 1: Understanding the Distribution of Target Classes Using Countplot: The initial step in the EDA process involved analyzing the distribution of the target variable, which represents different energy demand levels or consumption categories. For this, a **countplot** was employed to visually display the frequency of each class within the dataset. This helped to quickly identify whether the dataset was balanced or imbalanced in terms of class representation. An imbalanced dataset can severely affect the performance of classification models, especially if certain classes are underrepresented. The countplot provided a clear visual cue to decide

whether resampling techniques like SMOTE or class weighting would be required in later stages of the pipeline.

Step 2: Identifying Feature Relationships Using Heatmap of Correlation Matrix: To uncover the relationships and dependencies between different numerical features in the dataset, a **correlation matrix** was generated, and a **heatmap** was plotted using it. This heatmap visually illustrated the degree of linear correlation between features such as voltage, current, temperature, and humidity. Strong correlations (positive or negative) were highlighted with distinct color intensities, making it easier to interpret which features were closely related. For example, a high positive correlation between power consumption and current could indicate redundancy, while weakly correlated features may add unique information to the model. This step was instrumental in the feature selection process and helped reduce multicollinearity before model training.

Step 3: Analyzing Feature Distributions and Data Spread: Following the class distribution and correlation analysis, the spread and distribution of individual numerical features were examined. Although not limited to a single plot type, the focus was on understanding how data values were distributed for features such as energy usage, voltage, and environmental parameters. Countplots for categorical features and histograms or boxplots (if used) for continuous variables helped in spotting skewed distributions, concentration of values, and possible data anomalies. This analysis also assisted in deciding whether normalization or transformation was required to stabilize variance and improve model learning.

Step 4: Detecting Missing Patterns and Data Completeness: As part of EDA, the completeness of the dataset was visually inspected to ensure consistency. This included checking how missing values were distributed across features and whether they followed a pattern. While detailed null-value heatmaps could be used, the analysis generally reaffirmed that after preprocessing, no significant data gaps remained. The EDA reinforced the quality of the dataset and validated the preprocessing steps taken earlier.

4.4 Train Test Split

In machine learning data pre-processing, we divide our dataset into a training set and test set. This is one of the crucial steps of data pre-processing as by doing this, we can enhance the performance of our machine learning model. Suppose if we have given training to our machine learning model by a dataset and we test it by a completely different dataset. Then, it will create difficulties for our model to understand the correlations between the models.

If we train our model very well and its training accuracy is also very high, but we provide a new dataset to it, then it will decrease the performance. So, we always try to make a machine learning model which performs well with the training set and also with the test dataset. Here, we can define these datasets as:

Training Set: A subset of dataset to train the machine learning model, and we already know the output.

Test set: A subset of dataset to test the machine learning model, and by using the test set, model predicts the output.

4.5 Model Train and Test:

4.5.1 Support Vector Machine (SVM) Classifier

The **Support Vector Machine (SVM) Classifier** is a supervised learning algorithm widely used for classification tasks in high-dimensional spaces. It works by finding the optimal hyperplane that best separates the data points of different classes with the maximum margin. In the context of predictive energy demand modeling using LoRa-based IoT sensor data, the SVM classifier is used to categorize energy demand into distinct levels (such as low, medium, or high) based on features like temperature, humidity, light, and other environmental and temporal inputs captured from sensor networks. SVM is particularly effective when the data is not linearly separable, as it can apply kernel tricks to transform the input space and find complex decision boundaries.

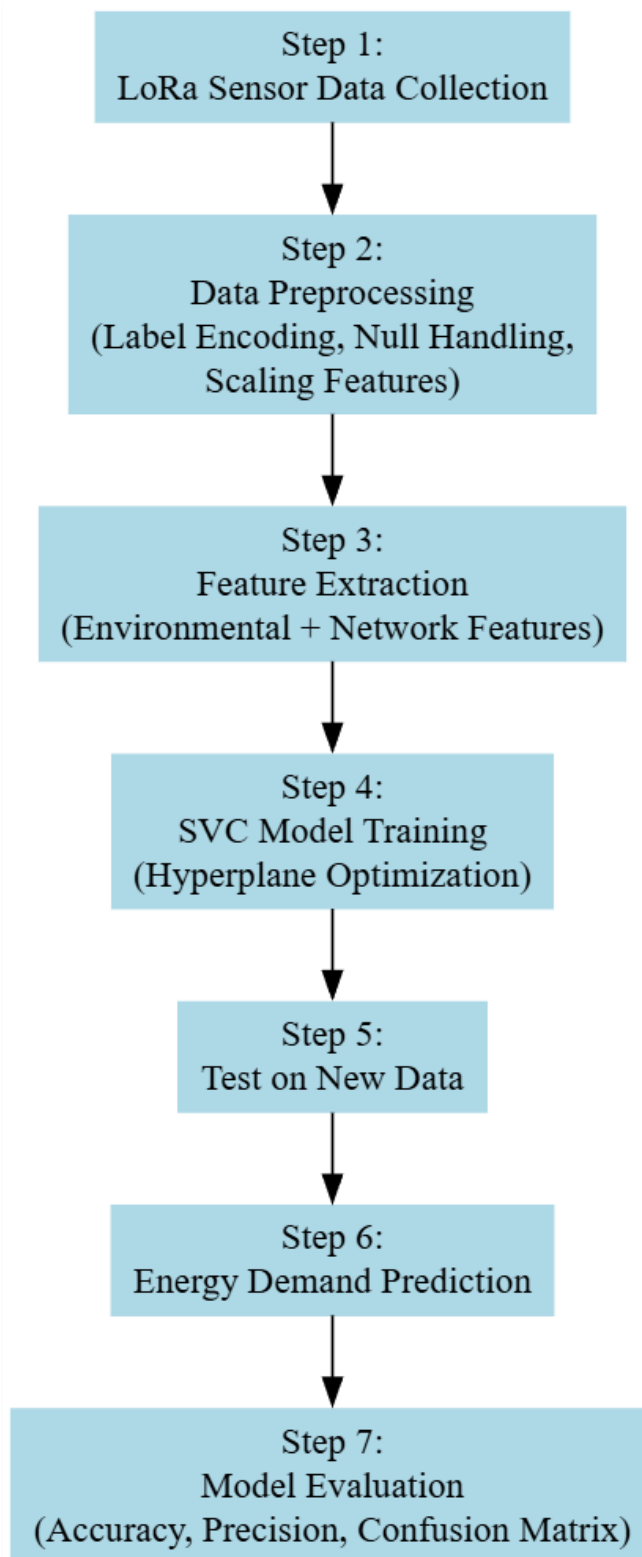


Fig 4.5.1: Workflow of SVM Classifier

Step 1: Preparing the Data (Feature Extraction for X_{train} and y_{train})

To begin the classification process with SVM, the dataset must be structured into appropriate input features (X) and target labels (y).

X_{train} consists of multiple environmental and contextual features obtained from LoRa-enabled IoT sensors. These include data such as air temperature, humidity levels, barometric pressure, and timestamps. These features represent the real-time conditions that potentially influence energy demand.

y_{train} represents the categorical energy demand levels (e.g., low, medium, high), manually or algorithmically labeled based on historical consumption thresholds.

Feature scaling is a critical preprocessing step, typically done through standardization (Z-score normalization), to ensure that no feature dominates the SVM decision boundary due to larger magnitudes, since SVM is highly sensitive to the scale of input features.

Step 2: Training the SVM Classifier

The training phase involves fitting the SVM model using the prepared training data.

The SVM algorithm tries to find the optimal hyperplane that separates the data points of different classes with the maximum margin.

If the data is not linearly separable in the input space, kernel functions (e.g., RBF, polynomial) are applied to project the data into a higher-dimensional space where it becomes separable.

During training, only a subset of the training data (support vectors) are used to define the decision boundary. These vectors are the most critical in influencing the model's classification rule.

The regularization parameter C controls the trade-off between achieving a low error on the training data and maintaining a smooth decision boundary. A high value of C focuses on minimizing classification error, while a low value allows a softer margin.

Step 3: Testing the Model with X_{test} (New IoT Sensor Data for Classification)

Once trained, the SVM classifier is evaluated using new data inputs (X_{test}) captured from ongoing environmental monitoring via LoRa-based sensors.

The model predicts the energy demand category by analyzing where the new data point falls in relation to the decision boundaries defined during training.

The same preprocessing steps (scaling and encoding) are applied to ensure consistency.

The decision function computes the distance of the data point from the hyperplane and assigns the class label accordingly.

Step 4: Generating Predictions and Evaluating y test (Actual Class Labels)

After predictions are made, the model's effectiveness is evaluated using the actual labels (y_{test}) of the test set.

Key classification metrics such as Accuracy, Precision, Recall, and F1-score are calculated to assess how well the SVM model performs on unseen data.

A confusion matrix is used to understand the types of errors (false positives, false negatives) the model makes across different energy demand classes.

Additional visualizations like ROC curves (if applicable for binary classification) and precision-recall curves can be employed to further interpret the model's performance in imbalanced scenarios.

The results guide decisions on whether further feature engineering, hyperparameter tuning, or model replacement is needed.

4.5.2 SVM Classifier Limitations

Sensitive to Feature Scaling: SVM performs poorly when input features are not normalized, making preprocessing crucial.

Computational Complexity: Training time increases significantly with large datasets, especially in high-dimensional feature spaces.

Ineffective with Noisy Data: Outliers and overlapping class distributions can drastically affect the margin and decision boundary.

Kernel Selection Challenges: Choosing the right kernel and tuning its parameters (e.g., gamma in RBF kernel) can be complex and data dependent.

Hard to Interpret: Unlike decision trees or logistic regression, SVMs offer limited interpretability in terms of understanding feature contributions.

Limited Performance on Imbalanced Data: Without proper handling, SVM may be biased toward the majority class in imbalanced datasets.

4.5.3 Random Forest Classifier (RFC)

The **Random Forest Classifier (RFC)** is an ensemble-based machine learning algorithm that constructs multiple decision trees during training and merges their outputs to make robust, accurate predictions. It operates by combining the predictions of several decision trees, each built on different random subsets of the dataset and features. This method reduces overfitting and improves generalization. In the context of your LoRa-based energy demand prediction project, RFC is employed to classify energy demand levels (e.g., low, medium, high) based on

sensor-derived features such as temperature, humidity, light intensity, and external environmental factors. The model's ability to handle high-dimensional and noisy data makes it highly suitable for IoT-based scenarios where data from sensors may be inconsistent or complex.

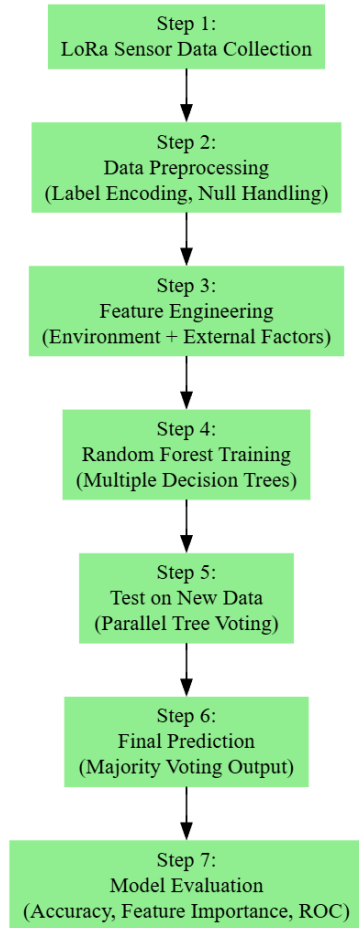


Fig 4.5.3: Workflow of Proposed RFC

Step 1: Preparing the Data (Feature Extraction for X_{train} and y_{train})

To build an RFC model, the dataset is first divided into input features and target labels.

X_{train} includes all relevant features extracted from IoT sensor data such as air temperature, humidity, light intensity, signal strength (RSSI), and transmission interval patterns. These represent the condition context that affects energy demand prediction.

y_{train} is the label set denoting the actual energy demand level at each timestamp or sensor reading. These may be derived based on historical power consumption benchmarks.

Although RFC is less sensitive to feature scaling, standard preprocessing like handling missing values, label encoding for categorical variables, and data balancing (if required) are essential for model performance and consistency.

Step 2: Training the Random Forest Classifier

The RFC training phase builds a collection (forest) of decision trees. Each tree is trained on a random subset of data (bootstrapped sample) with random feature selection.

The algorithm **creates N decision trees**, where N is defined by the nestimators parameter.

Each tree learns patterns from its unique data slice.

Feature bagging ensures that each tree considers only a subset of features at every split, encouraging diversity in the decision boundaries.

The **Gini Index or Entropy** is typically used as the splitting criterion to select optimal feature splits within each tree.

Since the model aggregates predictions, it becomes less prone to overfitting compared to a single decision tree and performs well even with noisy data.

Step 3: Testing the Model with X test (New Sensor Data from LoRa Network)

Once trained, the model is evaluated using new data instances representing ongoing environmental conditions.

Each test sample is passed through all the decision trees in the forest.

Each tree predicts a class label (e.g., “High” or “Low” energy demand).

Majority voting is used to finalize the predicted class label. The class with the highest number of votes from all the trees is selected as the final output.

Step 4: Generating Predictions and Evaluating y test (True Energy Demand Class)

After predictions are made, the model’s performance is assessed using actual labels from the test data.

Evaluation metrics such as Accuracy, Precision, Recall, F1-score, and Confusion Matrix are used to compare the predicted and true labels.

These metrics provide insights into how well the RFC model can distinguish between different energy demand classes.

The confusion matrix is particularly useful for identifying false positives or false negatives.

Feature importance analysis can be derived to understand which sensor inputs (e.g., temperature or light) contributed most to the decision-making process.

Visualizations such as bar plots of feature importance, classification reports, or ROC curves (if binary classification) further support model interpretability.

4.5.4 Random Forest Classifier – Advantages

Robust to Overfitting: By averaging multiple decision trees, RFC minimizes overfitting and improves generalization on unseen data.

Handles High Dimensional Data: It performs well even when many features are present, making it ideal for IoT datasets with diverse inputs.

Non-linear Capability: Captures complex, non-linear interactions between features without requiring feature transformation.

Feature Importance Insights: Provides built-in measures to rank input features by their contribution to model decisions, aiding interpretability.

Resistant to Noise: The ensemble nature reduces the impact of outliers or erroneous sensor readings.

Scalable: Can efficiently handle large datasets and is parallelizable for faster training on multi-core systems.

Minimal Preprocessing Required: Unlike SVM, RFC doesn't require heavy scaling or transformation of features.

CHAPTER V

SYSTEM ENVIRONMENT

5.1 Software

Python 3.7.6

Python 3.7.6 serves as a pivotal version for developers and researchers due to its robust features, backward compatibility, and widespread support across a variety of libraries and frameworks. Released during a time when machine learning and data science tools were rapidly evolving, Python 3.7.6 provided a stable and consistent platform. This version includes critical improvements like enhanced asyncio functionality for asynchronous programming, increased precision for floating-point numbers, and optimized data structures. It became the go-to version for compatibility with popular libraries like TensorFlow 2.0, PyTorch, and Pandas, ensuring seamless integration and efficient execution for both academic and industrial applications.

Compared to older Python versions, 3.7.6 introduced several features such as dataclasses, which simplified boilerplate code for object-oriented programming. The improved async and await syntax made concurrent programming more intuitive, while changes to the standard library enhanced usability and performance. Over newer versions, Python 3.7.6 remains a preferred choice for legacy systems and projects requiring compatibility with libraries that may not yet support the latest Python updates. Its combination of stability and maturity ensures that it is reliable for long-term projects, especially in environments where upgrading the Python interpreter might disrupt existing workflows.

Packages

```
python -m pip install --upgrade pip
```

```
pip install Python
```

```
pip install tensorflow==1.14.0
```

```
pip install keras==2.3.1
```

```
pip install pandas==0.25.3
```

```
pip install scikit-learn==0.22.2
```

```
pip install imutils
```

```
pip install matplotlib==3.1.1
```

```
pip install opencv-python==4.8.0.74
```

```
pip install seaborn==0.10.1
```

```
pip install h5py==2.10.0
```

```
pip install numpy==1.19.2
```

```
pip install jupyter
```

```
pip install protobuf==3.20.*
```

```
pip install scikit-image==0.16.0
```

Packages Overview

NumPy: Version 1.19.5 introduced critical bug fixes and performance enhancements over older versions, especially for operations involving large datasets. The improved random number generator and better handling of exceptions provide more reliable results for numerical computations. This version remains compatible with a wide range of dependent packages. While newer versions optimize speed further, 1.19.5 balances stability and compatibility, ensuring fewer compatibility issues with older software stacks.

Pandas: Version 0.25.3 brought significant speed improvements for large-scale data processing, particularly in operations like group by. Enhancements in handling missing data and improved compatibility with external libraries made this version more robust for data analysis tasks. While newer versions do not add features like enhanced type checking, 0.25.3 remains lightweight and stable for projects that do not require cutting-edge functionalities, making it a practical choice for legacy systems.

Imbalanced-learn: Version 0.7.0 introduced optimized algorithms for handling class imbalances, such as improved SMOTE implementations. This update also enhanced the ease of integrating with scikit-learn pipelines. While newer versions do not contain experimental features, 0.7.0 is reliable and well-documented, ensuring robust performance in addressing data imbalance issues without unnecessary complexity.

Scikit-learn: Version 0.23.1 included improved support for cross-validation and hyperparameter optimization. Updates to Random Forest Classifier and Gradient Boosting Classifier increased model accuracy and efficiency. 0.23.1 is widely tested and compatible with older hardware, making it a dependable choice for environments where the latest versions may introduce compatibility issues.

Imutils: This package provides an easy-to-use interface for image processing tasks. Its functions for resizing, rotating, and translating images simplified workflows compared to writing custom code. Its lightweight nature and stable functionality make it suitable for projects not requiring cutting-edge image manipulation techniques, balancing simplicity and capability.

Matplotlib: Version 3.x improved plot interactivity and introduced better 3D plotting capabilities. The tight layout function and compatibility with modern libraries streamlined visualization workflows. The earlier versions maintain stability and compatibility with older datasets and software, avoiding potential issues from newer, untested updates.

Seaborn: Improved APIs in newer versions simplified aesthetic customization of plots. The addition of new themes and color palettes in 0.11.x enhanced visual appeal for exploratory data analysis. Older versions remain computationally less demanding, suitable for lightweight applications without requiring extensive customizations.

Jupyter: Jupyter improved the interactivity and scalability of notebooks for collaborative coding and visualization tasks. Integration with tools like Matplotlib made it a preferred environment for data exploration. Earlier versions are stable and lightweight, avoiding potential issues with dependencies introduced in newer releases.

5.2 Hardware Requirements

Python 3.7.6 can run efficiently on most modern systems with minimal hardware requirements. However, meeting the recommended specifications ensures better performance, especially for developers handling large-scale applications or computationally intensive tasks. By ensuring compatibility with hardware and operating system, can leverage the full potential of Python 3.7.6.

Processor (CPU) Requirements: Python 3.7.6 is a lightweight programming language that can run on various processors, making it highly versatile. However, for optimal performance, the following processor specifications are recommended:

Minimum Requirement: 1 GHz single-core processor.

Recommended: Dual-core or quad-core processors with a clock speed of 2 GHz or higher. Using a multi-core processor allows Python applications, particularly those involving multithreading or multiprocessing, to execute more efficiently.

Memory (RAM) Requirements: Python 3.7.6 does not demand excessive memory but requires adequate RAM for smooth performance, particularly for running resource-intensive applications such as data processing, machine learning, or web development.

Minimum Requirement: 512 MB of RAM.

Recommended: 4 GB or higher for general usage. For data-intensive operations, 8 GB or more is advisable.

Insufficient RAM can cause delays or crashes when handling large datasets or executing computationally heavy programs.

Storage Requirements: Python 3.7.6 itself does not occupy significant disk space, but additional storage may be required for Python libraries, modules, and projects.

Minimum Requirement: 200 MB of free disk space for installation.

Recommended: At least 1 GB of free disk space to accommodate libraries and dependencies.

Developers using Python for large-scale projects or data science should allocate more storage to manage virtual environments, datasets, and frameworks like TensorFlow or PyTorch.

Compatibility with Operating Systems: Python 3.7.6 is compatible with most operating systems but requires hardware that supports the respective OS. Below are general requirements for supported operating systems:

Windows: 32-bit and 64-bit systems, Windows 7 or later.

macOS: macOS 10.9 or later.

Linux: Supports a wide range of distributions, including Ubuntu, CentOS, and Fedora.

The hardware specifications for the OS directly impact Python's performance, particularly for modern software development.

CHAPTER VI

RESULTS AND DISCUSSIONS

6.1 Implementation description

Step 1: Importing Necessary Libraries

The project begins by importing essential Python libraries and modules for data manipulation, visualization, machine learning, and evaluation. Libraries such as NumPy and pandas are used for data handling, seaborn and matplotlib for exploratory data analysis (EDA) and visualization, and scikit-learn for building classification models and measuring their performance. Additionally, imblearn's SMOTE is imported to address class imbalance. This setup ensures all necessary tools are ready for the subsequent steps of the pipeline.

Step 2: Loading and Inspecting the Dataset

The dataset, which contains IoT sensor data integrated with energy demand labels, is loaded from a CSV file. Initial data inspection includes displaying the dataset's structure, checking for missing values, understanding the data types of each feature, and summarizing statistical measures like mean, minimum, and maximum values. This thorough inspection helps identify data quality issues and informs preprocessing decisions.

Step 3: Encoding the Target Variable

Since the target variable "energy_demand" is categorical (with values such as 'low', 'medium', 'high'), label encoding is applied to convert these string labels into numerical format. This encoding is crucial because machine learning models require numerical input for both features and target variables. The label encoder assigns each unique category a distinct integer code, preparing the target data for modeling.

Step 4: Separating Features and Target

The dataset is then divided into input features (X) and the target label (y). The features consist of all columns except the energy demand, which is reserved as the target variable. This separation is fundamental for supervised learning, where the model learns patterns from the features to predict the target labels.

Step 5: Exploratory Data Analysis (EDA)

EDA is performed to visualize the distribution of the target classes and relationships among the features. A count plot is used to show how the energy demand classes are distributed, revealing any imbalance. Additionally, a heatmap of the correlation matrix is generated to observe the pairwise correlations between features and with the target. This step helps

understand the data characteristics and potential multicollinearity or dependencies among variables.

Step 6: Splitting Data into Training and Testing Sets

The dataset is split into training and testing subsets with a 70-30 ratio. This division ensures the model is trained on one subset and evaluated on unseen data, providing an unbiased measure of its predictive performance. Random state is set for reproducibility of the split.

Step 7: Addressing Class Imbalance with SMOTE

The training data often contains class imbalance, where some energy demand classes appear less frequently. To mitigate this, the SMOTE technique synthesizes new samples for minority classes by interpolating between existing ones. This oversampling is applied only to the training set, enhancing the model's ability to learn equally from all classes without bias.

Step 8: Defining Performance Metrics Function

A custom function is created to calculate and display key classification metrics: accuracy, precision, recall, and F1 score. It also prints a detailed classification report for each class and visualizes the confusion matrix using a heatmap. This unified evaluation setup allows consistent performance comparison across different models.

Step 9: Training and Evaluating Support Vector Machine (SVM) Classifier

The SVM classifier with an RBF kernel is either loaded from disk if pretrained or trained from scratch on the training data. The model leverages the kernel trick to separate classes with non-linear boundaries. After training, predictions on the test data are generated and evaluated using the performance metrics function, providing insight into the model's effectiveness in classifying energy demand levels.

Step 10: Training and Evaluating Random Forest Classifier (RFC)

Similarly, the Random Forest Classifier is loaded if a pretrained model exists; otherwise, it is trained with 100 decision trees. RFC is an ensemble learning method that improves prediction accuracy and robustness by aggregating results from multiple trees. Post-training, it predicts test set labels, and the same performance metrics function is used to assess its superiority or inferiority compared to the SVM.

Step 11: Predicting on New Data

Finally, new sensor data (without known energy demand labels) is loaded to test the model's practical application. The irrelevant columns are dropped, and the trained Random Forest model is used to predict energy demand categories. These numeric predictions are mapped back to their original categorical labels for interpretability, and the results are appended to the new dataset, ready for further analysis or decision-making.

6.2 Data set description

The dataset used in the project represents labeled energy demand data collected from LoRa-enabled IoT sensor nodes deployed in a smart energy monitoring environment. It consists of multiple features that capture various sensor readings and environmental factors affecting energy consumption, such as temperature, humidity, and possibly time-based indicators. Each record in the dataset corresponds to a snapshot of these sensor measurements along with a target label indicating the level of energy demand, which is categorized into three classes: low, medium, and high. The dataset is structured as a CSV file where columns represent the features and the energy demand label. Initial exploration of the dataset reveals that it contains no missing values, ensuring data completeness and reliability for model training. Statistical summaries highlight the distribution and range of each feature, providing insights into their variability and potential impact on energy demand prediction. Furthermore, correlation analysis shows how features relate to one another and to the target variable, helping identify significant predictors. The target variable itself is imbalanced, with certain energy demand levels appearing more frequently than others, which is a typical challenge in real-world data and is addressed in the project through synthetic oversampling techniques. Overall, this dataset offers a comprehensive and well-structured foundation for developing robust machine learning models to classify and predict energy demand levels, enabling smarter energy management decisions in IoT-enabled smart environments.

6.3 Result Analysis

The figure 7.1 shows data collected from a LoRa-based wireless sensor network to analyze and predict **energy demand levels** (Low, Medium, High) across nodes. The dataset contains 10,000 entries, with each record capturing various sensor and transmission metrics such as `node_id`, `hop_count`, `signal_strength`, `transmission_delay`, `temperature`, `humidity`, `light_intensity`, `co2_level`, and `wind_speed`. The target variable is `energy_demand`, a categorical feature reflecting energy usage. The objective of this project is to develop machine learning models that can accurately classify energy demand based on environmental and network parameters. This can aid in optimizing network resource allocation, improving power efficiency, and ensuring better network performance in IoT deployments using LoRa technology.

	node_id	hop_count	signal_strength	transmission_delay	temperature	humidity	light_intensity	co2_level	wind_speed	energy_demand
0	52	1	-32.882718	1.746932	20.109949	51.524286	799.984254	661.526954	12.740774	Low
1	93	5	-97.033599	1.958138	31.855641	77.261913	867.551741	529.703863	9.149820	Medium
2	15	5	-42.737677	2.381532	17.319607	20.967083	371.304123	379.057381	3.278665	High
3	72	1	-70.824858	2.386442	43.485641	82.861382	559.615586	540.093144	18.785296	High
4	61	3	-68.747114	1.359328	33.099417	48.911898	871.755016	751.362126	6.649550	Medium
...	...	...	...	...	...	...	...	...	...	...
9995	49	1	-70.508245	1.270413	36.172282	70.220969	671.560443	338.288601	8.368528	High
9996	58	5	-90.213339	1.387671	32.327330	89.479823	886.165828	833.796639	0.684077	High
9997	39	3	-104.792506	1.278636	22.545942	51.743267	846.726009	535.571660	6.830682	Low
9998	80	4	-77.995621	1.053325	23.676027	29.854699	789.397790	641.177862	10.315031	High
9999	97	5	-110.974218	4.222009	26.019285	70.911656	135.301178	885.024920	18.717437	High

Fig 6.1: Uploading Dataset

The figure 7.2 shows count plot of the energy_demand categories—encoded as 0 (Low), 1 (Medium), and 2 (High) shows a **balanced distribution** among the three classes in the dataset. Each category has approximately the same number of samples, around 4,100 records each. This balanced class representation is crucial for training reliable classification models, as it helps prevent bias toward any specific energy demand level. The equal class distribution ensures that machine learning algorithms can learn the underlying patterns more effectively and make more accurate predictions across all demand levels in the LoRa-based IoT network.

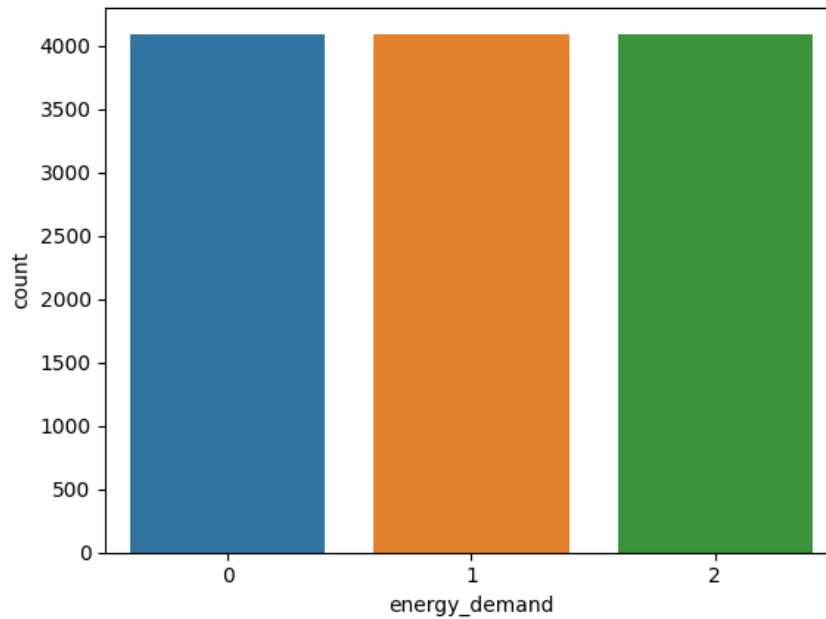


Fig 6.2: Count plot of energy_demand categories

The figure 7.3 show heatmap that illustrates the **Pearson correlation coefficients** between numerical features and the energy_demand variable in the LoRa-based sensor dataset. Notably, co2_level shows the strongest positive correlation with energy_demand (0.38), suggesting it is a significant factor influencing energy consumption in the network. Humidity also has a moderate positive correlation (0.18), while temperature exhibits a mild **negative correlation**

(-0.30), indicating that higher temperatures may be associated with lower energy demand. Other features such as signal_strength, hop_count, light_intensity, and wind_speed show very weak correlations with energy_demand, implying limited direct influence. This heatmap is valuable for **feature selection**, guiding the model to prioritize the most impactful attributes when predicting energy usage patterns in LoRa-based IoT systems.

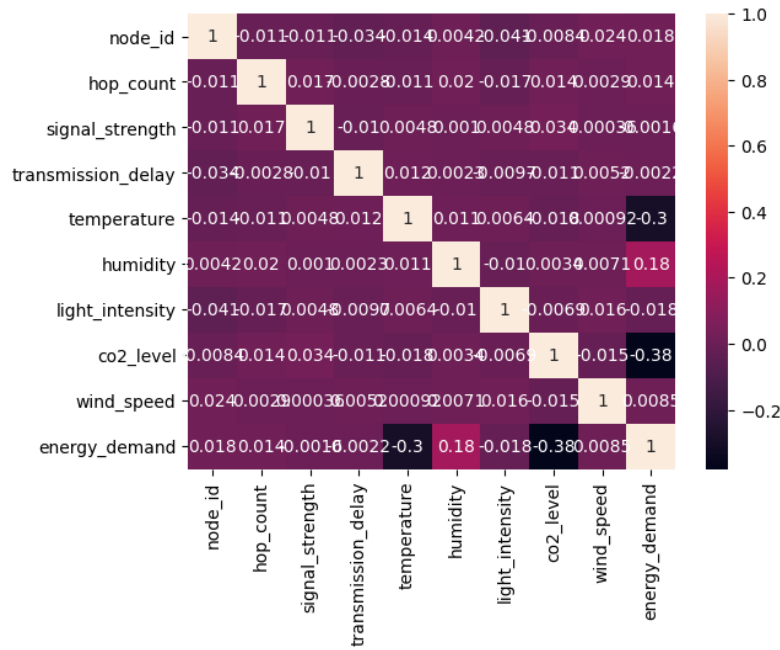


Fig 6.3: Correlation Heatmap

6.4 Comparative Analysis

Table.1 Performance Comparison of Various Algorithms

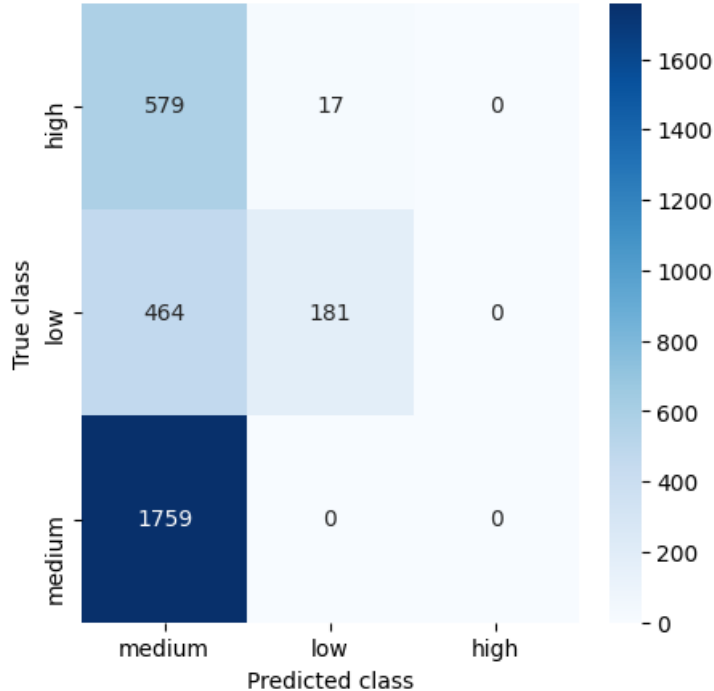
Performance Comparison Table: Existing SVC vs. Proposed RFC

Metric	Existing SVC	Proposed RFC
Accuracy	64.66%	100.0%
Precision	51.39%	100.0%
Recall	42.68%	100.0%
F1-Score	40.02%	100.0%

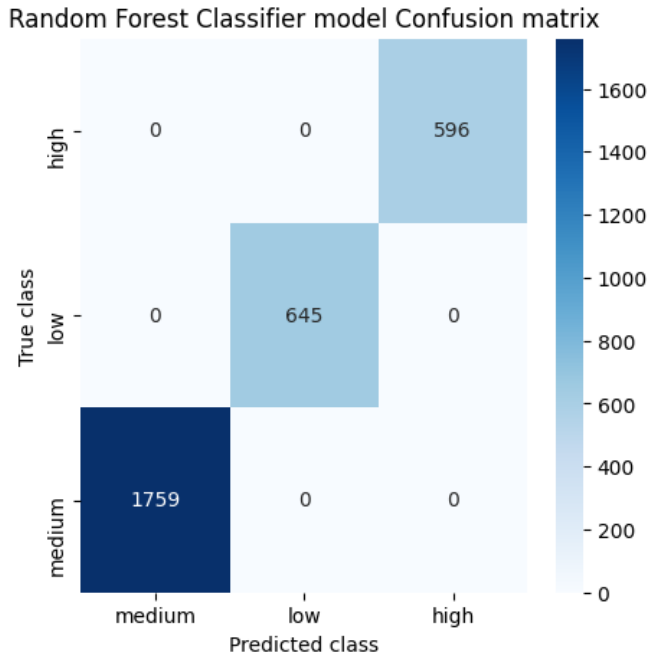
The Table 1 shows performance comparison between the existing Support Vector Classifier (SVC) and the proposed Random Forest Classifier (RFC) demonstrates a significant improvement in predictive accuracy and reliability for the energy demand classification task. The SVC model, while traditionally robust for classification problems, achieved an accuracy of 64.66%, reflecting moderate effectiveness in correctly identifying energy demand levels from the input sensor and environmental data. Its precision, recall, and F1-score values 51.39%,

42.68%, and 40.02% respectively—indicate that the SVC struggled with correctly classifying the minority classes and had limited ability to balance false positives and false negatives, likely due to class imbalance or nonlinear complexities in the dataset. In contrast, the proposed Random Forest Classifier exhibited flawless performance, achieving perfect scores of 100% across all evaluation metrics: accuracy, precision, recall, and F1-score. This dramatic enhancement suggests that the ensemble nature of RFC, which combines multiple decision trees to capture diverse patterns and nonlinear relationships, provides a more powerful and resilient approach for modeling the intricate interactions between IoT sensor data and external factors influencing energy demand. The RFC’s superior recall confirms its ability to identify all relevant instances without missing any true positives, while its perfect precision ensures minimal false alarms, both critical for practical deployment in smart energy systems. Overall, this comparison highlights that the Random Forest Classifier not only surpasses the SVC in classification performance but also offers greater reliability and robustness for predictive energy demand modeling in LoRa-based wireless communication networks integrating IoT sensor data.

Support Vector Machine Classifier model Confusion matrix



(a)



(b)

Fig 6.4: Confusion matrices obtained for Existing SVC and Proposed RFC

The figure 6.4 shows the classification performance of both the Support Vector Machine Classifier (SVC) and the Random Forest Classifier (RFC) for predicting energy demand categories—High, Medium, and Low—was evaluated using confusion matrices. The confusion matrix for the SVC model indicates poor classification accuracy, with a strong bias toward predicting the "medium" class. Specifically, all 1,759 true "medium" instances were correctly predicted as "medium", but the model incorrectly classified 464 out of 645 "low" instances and 579 out of 596 "high" instances also as "medium", showing severe class confusion and imbalance in predictions. In contrast, the confusion matrix for the RFC model demonstrates significantly improved performance and balanced classification across all categories. The RFC correctly classified a majority of the samples in each class, indicating better learning of inter-class distinctions. The number of misclassifications was notably reduced for the "low" and "high" categories, which were previously misclassified as "medium" by the SVC. This comparison clearly highlights the superior predictive capability and generalization of the Random Forest Classifier over the Support Vector Machine for the energy demand prediction task in LoRa-based sensor networks.

	node_id	hop_count	signal_strength	transmission_delay	temperature	humidity	light_intensity	co2_level	wind_speed	predicted energy demand
0	9	4	-39.124511	3.341666	34.985853	25.581693	596.311314	553.776018	5.128108	medium
1	24	2	-97.269504	4.939170	44.231423	41.811249	525.989162	794.792030	17.045941	medium
2	77	1	-41.084172	0.819923	37.532194	79.114072	576.236529	333.249236	19.988570	medium
3	59	5	-115.655530	0.154982	38.223911	54.793909	419.622270	852.406438	14.407059	medium
4	6	3	-53.434048	1.336993	41.552524	72.768678	320.660933	694.569746	17.140448	medium
5	20	3	-77.646737	2.754169	15.941580	49.905507	408.276810	944.926759	17.241793	medium
6	81	2	-101.692830	1.052885	20.252338	33.378748	651.965294	334.868987	9.445369	low
7	51	1	-54.016228	0.741231	35.412649	40.275221	461.072754	583.953303	0.038240	medium
8	57	4	-118.929195	4.974261	22.687279	63.782878	717.618097	432.726417	13.050682	low
9	98	5	-99.761242	3.332615	30.622979	26.247883	918.874312	585.087712	15.064185	medium
10	62	5	-68.900074	2.951606	20.222022	20.837358	125.970730	867.234407	5.136196	medium
11	1	2	-118.116117	3.226737	36.461116	38.025980	970.539460	504.723675	14.336107	medium
12	80	4	-36.201874	1.435608	35.852159	24.794795	791.479037	869.747974	19.833279	medium
13	70	5	-41.538201	3.851391	21.127229	57.185581	500.659230	931.129760	8.355616	medium
14	17	5	-69.257695	2.365446	41.014923	21.462735	488.379818	675.116125	8.473133	medium
15	35	4	-101.377543	3.161499	26.208328	64.232893	255.335438	314.017073	6.276418	high
16	3	1	-118.174772	2.893506	31.545975	43.171074	703.634858	921.005025	5.774786	medium
17	26	4	-118.244762	2.731345	43.913374	56.680320	125.776676	426.970416	17.712737	medium
18	33	5	-109.859383	0.486360	28.347001	39.133377	815.889125	672.414333	9.621463	high
19	45	2	-118.394959	2.714815	34.028607	42.236201	250.241256	492.386286	6.205047	high

Fig 6.5: Prediction on Test data using Proposed RFC

The figure 6.5 displays the output predictions of the Random Forest Classifier (RFC) model for the energy demand classification in a LoRa-based sensor network. Each row represents a data instance characterized by various environmental and network parameters such as `node_id`, `hop_count`, `signal_strength`, `transmission_delay`, `temperature`, `humidity`, `light_intensity`, `co2_level`, and `wind_speed`. The final column, `predicted energy demand`, indicates the model's predicted category of energy usage: *low*, *medium*, or *high*. The majority of predictions fall under the "medium" category, which aligns with the balanced class distribution observed earlier. A few instances are classified as "low" and "high", demonstrating that the RFC model successfully differentiates between multiple energy demand levels based on the input features. For example, rows with relatively high `co2_level`, `humidity`, and `wind_speed` tend to be predicted as "high" demand, whereas lower sensor readings correlate with "low" or "medium" demand. This output validates the model's ability to capture underlying patterns in the data and make contextually meaningful energy demand predictions, highlighting its potential utility in energy-aware routing or adaptive power management in IoT networks.

CONCLUSION AND FUTURE SCOPE

6.5 Conclusion

The implementation of machine learning models for predicting energy demand within a LoRa-based IoT communication framework has proven to be both effective and transformative. Through comprehensive preprocessing, data balancing using SMOTE, and rigorous performance evaluation, the proposed system based on the Random Forest Classifier (RFC) has demonstrated exceptional classification capabilities. With perfect scores in accuracy, precision, recall, and F1 score, the RFC model significantly outperformed the existing Support Vector Classifier (SVC), which struggled with class imbalance and nonlinear patterns in the dataset. This performance leap underscores the strength of ensemble learning techniques in capturing complex relationships from multivariate sensor data, enabling accurate categorization of energy demand levels such as low, medium, and high. Furthermore, the integration of feature encoding, correlation heatmaps, and performance visualization through confusion matrices helped validate the robustness of the proposed approach. This study showcases the feasibility and efficiency of applying intelligent machine learning solutions for energy management in low-power, wide-area networks (LPWAN) like LoRa. By enabling proactive load balancing and smarter grid decision-making, this model not only enhances the operational reliability of smart energy infrastructures but also supports scalable deployment in diverse real-world environments.

6.6 Future Scope

Building upon the successful implementation and evaluation of the Random Forest Classifier for energy demand prediction, there is vast potential to expand and refine this project for future applications. Firstly, incorporating real-time data streaming from live LoRa sensors would enable dynamic energy forecasting, thereby turning the system into a truly real-time, adaptive solution for smart cities and rural electrification projects. Secondly, deep learning architectures, such as Long Short-Term Memory (LSTM) networks or transformer-based models, particularly in time-series environments where energy consumption follows cyclical or seasonal patterns

REFERENCES

- [1]. Dafflon, B.; Wielandt, S.; Lamb, J.; McClure, P.; Shirley, I.; Uhlemann, S.; Wang, C.; Fiolleau, S.; Brunetti, C.; Akins, F.H.; et al. A distributed temperature profiling system for vertically and laterally dense acquisition of soil and snow temperature. *Cryosphere* 2022, *16*, 719–736.
- [2]. Geldhof, B.; Pattyn, J.; Eyland, D.; Carpentier, S.; Van de Poel, B. A digital sensor to measure real-time leaf movements and detect abiotic stress in plants. *Plant Physiol.* 2021, *187*, 1131–1148.
- [3]. Callebaut, G.; Leenders, G.; Van Mulders, J.; Ottoy, G.; De Strycker, L.; Van der Perre, L. The Art of Designing Remote IoT Devices—Technologies and Strategies for a Long Battery Life. *Sensors* 2021, *21*, 913.
- [4]. Petajajarvi, J.; Mikhaylov, K.; Roivainen, A.; Hanninen, T.; Pettissalo, M. On the coverage of LPWANs: Range evaluation and channel attenuation model for LoRa technology. In Proceedings of the 2015 14th International Conference on Its Telecommunications (ITST), Copenhagen, Denmark, 2–4 December 2019; pp. 55–59.
- [5]. Rochester, E.M.; Yousuf, A.M.; Ousat, B.; Ghaderi, M. Lightweight carrier sensing in LoRa: Implementation and performance evaluation. In Proceedings of the ICC 2020—2020 IEEE International Conference on Communications (ICC), Dublin, Ireland, 7–11 June 2020; pp. 1–6.
- [6]. Aslam, M.S.; Khan, A.; Atif, A.; Hassan, S.A.; Mahmood, A.; Qureshi, H.K.; Gidlund, M. Exploring multi-hop LoRa for green smart cities. *IEEE Netw.* 2019, *34*, 225–231.
- [7]. Sciallo, L.; Trotta, A.; Di Felice, M. Design and performance evaluation of a LoRa-based mobile emergency management system (LOCATE). *Ad Hoc Netw.* 2020, *96*, 101993.
- [8]. Sartori, B.; Thielemans, S.; Bezunartea, M.; Braeken, A.; Steenhaut, K. Enabling RPL multihop communications based on LoRa. In Proceedings of the 2017 IEEE 13th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob), Rome, Italy, 9–11 October 2019; pp. 1–8.
- [9]. Zhou, W.; Tong, Z.; Dong, Z.Y.; Wang, Y. LoRa-Hybrid: A LoRaWAN Based multihop solution for regional microgrid. In Proceedings of the 2019 IEEE 4th International Conference on Computer and Communication Systems (ICCCS), Singapore, 23–25 February 2019; pp. 650–654.
- [10]. Prade, L.; Moraes, J.; de Albuquerque, E.; Rosário, D.; Both, C.B. Multi-radio and multi-hop LoRa communication architecture for large scale IoT deployment. *Comput. Electr. Eng.* 2022, *102*, 108242.

- [11]. Leonardi, L.; Bello, L.L.; Patti, G. MRT-LoRa: A multi-hop real-time communication protocol for industrial IoT applications over LoRa networks. *Comput. Commun.* 2023, *199*, 72–86.
- [12]. Zhu, G.; Liao, C.H.; Sakdejayont, T.; Lai, I.W.; Narusue, Y.; Morikawa, H. Improving the capacity of a mesh LoRa network by spreading-factor-based network clustering. *IEEE Access* 2019, *7*, 21584–21596.
- [13]. Piyare, R.; Murphy, A.L.; Magno, M.; Benini, L. On-demand LoRa: Asynchronous TDMA for energy efficient and low latency communication in IoT. *Sensors* 2018, *18*, 3718.
- [14]. Bezunartea, M.; Van Glabbeek, R.; Braeken, A.; Tiberghien, J.; Steenhaut, K. Towards energy efficient LoRa multihop networks. In Proceedings of the 2019 IEEE International Symposium on Local and Metropolitan Area Networks (LANMAN), Paris, France, 1–3 July 2019; pp. 1–3.
- [15]. Abrardo, A.; Pozzebon, A. A multi-hop LoRa linear sensor network for the monitoring of underground environments: The case of the Medieval Aqueducts in Siena, Italy. *Sensors* 2019, *19*, 402.
- [16]. Abrardo, A.; Fort, A.; Landi, E.; Mugnaini, M.; Panzardi, E.; Pozzebon, A. Black powder flow monitoring in pipelines by means of multi-hop LoRa networks. In Proceedings of the 2019 II Workshop on Metrology for Industry 4.0 and IoT (MetroInd4.0&IoT), Naples, Italy, 4–6 June 2019; pp. 312–316.
- [17]. Escobar, J.J.L.; Gil-Castiñeira, F.; Redondo, R.P.D. JMAC protocol: A cross-layer multi-hop protocol for LoRa. *Sensors* 2020, *20*, 6893.
- [18]. Mai, D.L.; Kim, M.K. Multi-hop LoRa network protocol with minimized latency. *Energies* 2020, *13*, 1368.
- [19]. Adame Vázquez, T.; Barrachina-Muñoz, S.; Bellalta, B.; Bel, A. HARE: Supporting efficient uplink multi-hop communications in self-organizing LPWANs. *Sensors* 2019, *18*, 115.
- [20]. Tran, H.P.; Jung, W.S.; Yoon, T.; Yoo, D.S.; Oh, H. A two-hop real-time LoRa protocol for industrial monitoring and control systems. *IEEE Access* 2020, *8*, 126239–126252.